JavaScript is disabled on your browser.

Interface ForkJoinPool.ManagedBlocker

Enclosing class:

ForkJoinPool

```
public static interface ForkJoinPool.ManagedBlocker
```

Interface for extending managed parallelism for tasks running in ForkJoinPools.

A `ManagedBlocker` provides two methods. Method `isReleasable()` must return `true` if blocking is not necessary. Method `block()` blocks the current thread if necessary (perhaps internally invoking `isReleasable` before actually blocking). These actions are performed by any thread invoking `ForkJoinPool.managedBlock(ManagedBlocker)`. The unusual methods in this API accommodate synchronizers that may, but don't usually, block for long periods. Similarly, they allow more efficient internal handling of cases in which additional workers may be, but usually are not, needed to ensure sufficient parallelism. Toward this end, implementations of method `isReleasable` must be amenable to repeated invocation.

For example, here is a ManagedBlocker based on a ReentrantLock:

```
 class ManagedLocker implements ManagedBlocker {
   final ReentrantLock lock;
   boolean hasLock = false;
   ManagedLocker(ReentrantLock lock) { this.lock = lock; }
   public boolean block() {
     if (!hasLock)
       lock.lock();
     return true;
   }
   public boolean isReleasable() {
```

```
    return hasLock || (hasLock = lock.tryLock());
  }
}
```

Here is a class that possibly blocks waiting for an item on a given queue:

```
class QueueTaker<E> implements ManagedBlocker {
  final BlockingQueue<E> queue;
  volatile E item = null;
  QueueTaker(BlockingQueue<E> q) { this.queue = q; }
  public boolean block() throws InterruptedException {
    if (item == null)
      item = queue.take();
    return true;
  }
  public boolean isReleasable() {
    return item != null || (item = queue.poll()) != null;
  }
  public E getItem() { // call after pool.managedBlock completes
    return item;
  }
}
```

Method Summary

Abstract Methods

Modifier and Type	Method	Description
boolean	**block**()	Possibly blocks the current thread, for example waiting for a lock or condition.
boolean	**isReleasable**()	Returns `true` if blocking is unnecessary.

Method Detail

block

```
boolean block()
       throws InterruptedException
```

Possibly blocks the current thread, for example waiting for a lock or condition.

Returns:

true if no additional blocking is necessary (i.e., if isReleasable would return true)

Throws:

InterruptedException - if interrupted while waiting (the method is not required to do so, but is allowed to)

isReleasable

```
boolean isReleasable()
```

Returns true if blocking is unnecessary.

Returns:

true if blocking is unnecessary

Submit a bug or feature

For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.

Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms. Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.